



2.4 Threads (aspectos avançados)

Celso Paím



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.4.1 Executor Service

- O `ExecutorService` é um *framework* que simplifica o gerenciamento de tarefas assíncronas e pools de *threads* oferecendo um controle muito maior, permitindo configurar a quantidade de *threads*, definir quando criá-las ou destruí-las e como enfileirar as execuções.
- Está definida no pacote `java.util.concurrent`.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.4.1 Executor Service

- Principais vantagens:
 - ✓ Gerencia pools de threads, reutilizando um número fixo de threads em vez de criar novas a cada vez (ao contrário da classe `Thread`)
 - ✓ Executa tarefas `Runnable` (retorno void) e `Callable` (retorna um resultado reaproveitável)
 - ✓ Retorna resultados com `Future` (objecto que representa uma tarefa executando em background e em que o resultado só estará disponível no futuro)
 - ✓ Escala facilmente entre núcleos de CPU
- Principais desafios:
 - ✓ Complexidade no acompanhamento de tarefas, pois recuperar resultados ou gerenciar tarefas assíncronas requer o uso de `Future` e `Callable`, o que aumenta a complexidade do código em comparação com operações síncronas.
 - ✓ Tarefas enviadas ao `Executor` podem lançar exceções silenciosamente. Se não forem tratadas, a thread pode morrer e o erro passar despercebido.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.4.1 Executor Service

Principais métodos:

- **shutdown()**: desliga o *Executor* de forma ordenada; ele pára de aceitar novas tarefas (*threads*), mas finaliza as que já estão em processamento.
- **shutdownNow()**: tenta interromper as tarefas em execução imediatamente e retorna uma lista das tarefas que estavam na fila e não foram iniciadas.
- **invokeAll()**: executa uma coleção de tarefas e retorna uma lista de `Future` quando todas estiverem completas.
- **invokeAny()**: executa as tarefas e retorna o resultado da primeira que terminar com sucesso, cancelando as demais.
- **submit()**: envia uma tarefa e retorna um objecto `Future`, permitindo verificar o estado, obter o resultado ou cancelar a execução.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim, adaptado de Tafseer Mannan
// Data: Marco/2026
// Objectivo: demonstrar o uso de um ExecutorService

```
import java.util.concurrent.*;
```

```
public class ExecutorServiceDemo {  
    public static void main( String[] args ) {  
        ExecutorService executor = Executors.newFixedThreadPool( 3 );  
  
        Callable<String> task1 = () -> {  
            Thread.sleep( 5000 );  
            return "Resultado da tarefa 1";  
        };  
  
        Callable<String> task2 = () -> {  
            Thread.sleep( 500 );  
            return "Resultado da tarefa 2";  
        };  
    }  
}
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
Callable<String> task3 = () -> {  
    Thread.sleep( 700 );  
    return "Resultado da tarefa 3";  
};  
  
try {  
    Future<String> result1 = executor.submit( task1 );  
    Future<String> result2 = executor.submit( task2 );  
    Future<String> result3 = executor.submit( task3 );  
  
    System.out.println( result1.get() );  
    System.out.println( result2.get() );  
    System.out.println( result3.get() );  
}  
catch ( InterruptedException | ExecutionException e ) {  
    e.printStackTrace();  
} finally {  
    executor.shutdown();  
}  
}
```



2.4.2 Scheduled Executor Service

- O `ScheduledExecutorService` é uma interface do pacote `java.util.concurrent` que permite agendar tarefas para serem executadas após um atraso específico ou repetidamente em intervalos fixos.
- Também é uma alternativa mais robusta e flexível ao `sleep` das *threads* (que degrada performance colocando a *thread* em modo de ESPERA_SINCRONIZADA).
- Tem 3 formas de se usar:
 - `schedule` (executar uma vez depois do tempo indicado)
 - `scheduleAtFixedRate` (repete em intervalo fixo, mesmo se atrasar)
 - `scheduleWithFixedDelay` (espera acabar, adiciona um *delay* e volta a repetir)



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim, adaptado de Tafseer Mannan

// Data: Marco/2026

// Objectivo: demonstrar o uso de um ScheduledExecutorService que executa uma vez

```
import java.util.concurrent.*;
```

```
public class ScheduledExecutorRunnable {  
    public static void main( String[] args ) {
```

```
        ScheduledExecutorService ses = Executors.newScheduledThreadPool( 1 );
```

```
        Runnable task2 = () -> System.out.println( "Executando a tarefa 2..." );
```

```
        task1();
```

```
        ses.schedule( task2, 5, TimeUnit.SECONDS );
```

```
        task3();
```

```
        ses.shutdown();
```

```
    }
```




Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
public static void task1() {  
    System.out.println( "Executando a tarefa 1..." );  
}  
  
public static void task3() {  
    System.out.println( "Executando a tarefa 3..." );  
}  
}
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim, adaptado de Tafseer Mannan

// Data: Marco/2026

// Objectivo: demonstrar o uso de um ScheduledExecutorService que executa repetidas vezes

```
import java.util.concurrent.*;
```

```
public class ScheduledExecutorRepeat {
```

```
    private static int count = 0;
```

```
    public static void main( String[] args ) throws InterruptedException {
```

```
        ScheduledExecutorService ses = Executors.newScheduledThreadPool( 1 );
```

```
        Runnable task1 = () -> {
```

```
            count++;
```

```
            System.out.println( "Executando a tarefa 1 - count = " + count );
```

```
        };
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// iniciar com delay de 5s e repetir a tarefa a cada 1s

```
ScheduledFuture<?> scheduledFuture = ses.scheduleAtFixedRate( task1, 5, 1,  
TimeUnit.SECONDS );
```

```
while ( true ) {  
    System.out.println( "count = " + count );  
    Thread.sleep( 1000 );  
    if ( count == 5 ) {  
        System.out.println( "Count = 5, cancelando o scheduledFuture!" );  
        scheduledFuture.cancel( true );  
        ses.shutdown();  
        break;  
    }  
}  
}
```



2.4.3 ThreadPoolExecutor

É uma classe que permite gerir grupos de *threads* reutilizáveis de forma eficiente, sendo ideal para ambientes de alta concorrência e processamento em segundo plano, oferecendo uma vasta gama de configurações.

Para configurar um `ThreadPoolExecutor`, precisamos definir 5 parâmetros básicos:

- `corePoolSize`: # de *threads* principais a serem mantidas na *pool*, mesmo que estejam ociosas.
- `maximumPoolSize`: # máximo de *threads* permitidas na *pool* caso a demanda aumente drasticamente.
- `keepAliveTime`: determina quanto tempo as *threads* excedentes (criadas além do `corePoolSize`) devem esperar por novas tarefas antes de serem encerradas. Pode ser especificado em milissegundos, segundos, etc.).



2.4.3 ThreadPoolExecutor

- `workQueue`: a fila usada para armazenar tarefas antes de serem executadas. As tarefas são colocadas aqui quando todas as *threads* do `corePoolSize` estão ocupadas.
- `handler` (`RejectedExecutionHandler`): a política a ser usada quando uma tarefa não pode ser executada (ex: a *pool* e a fila estão cheios). Existem quatro políticas pré-definidas:

`AbortPolicy` (padrão): lança uma exceção ao rejeitar a tarefa.

`CallerRunsPolicy`: executa a tarefa na *thread* que chamou o *submit*, reduzindo a velocidade de novas submissões.

`DiscardPolicy`: descarta a nova tarefa silenciosamente.

`DiscardOldestPolicy`: descarta a tarefa mais antiga da fila e tenta resubmeter a nova.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim, adaptado de Pranav Tiwari

// Data: Marco/2026

// Objectivo: demonstrar o uso de um ScheduledExecutorService que executa uma vez

```
import java.util.concurrent.*;
```

```
public class ThreadPoolExecutorDemo {  
    public static void main( String[] args ) {  
        ThreadPoolExecutor executor = new ThreadPoolExecutor(  
            2, // core pool  
            4, // max pool  
            1, // keepalive time  
            TimeUnit.SECONDS,  
            new LinkedBlockingQueue<>() // queue de tarefas  
        );  
  
        // submeter 5 tarefas ao executor  
        for( int i = 1; i <= 5; i++ ) {  
            final int taskId = i;  
            executor.execute() -> {  
                System.out.println( "Tarefa " + taskId + " a ser executada pela " +  
                    Thread.currentThread().getName() );  
            }  
        }  
    }  
}
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
try {  
    Thread.sleep( 1000 );  
}  
catch ( InterruptedException e ) {  
    Thread.currentThread().interrupt();  
}  
System.out.println( "Tarefa " + taskId + " completada pela " +  
Thread.currentThread().getName() );  
});  
}  
  
executor.shutdown();  
}  
}
```



2.4.4 Virtual Threads

- As *threads* 'normais' ou de plataforma são geridas pelo SO e o seu uso incorreto pode causar problemas de segurança (*race conditions*), travamentos (*deadlocks*) e queda de performance devido ao excesso de troca de contexto (*context switch*). Esse cenário mudou drasticamente com o lançamento do Java 21 (2023), que introduziu as *Virtual Threads*, que são geridas directamente pela JVM.
- O funcionamento das *threads* virtuais é análogo ao da memória virtual: para simular uma grande quantidade de memória, um sistema operativo mapeia um vasto espaço de endereçamento virtual para uma quantidade limitada de RAM física. Da mesma forma, o ambiente de execução Java (JVM) mapeia um grande número de *threads* virtuais para um pequeno número de *threads* do sistema operacional (conhecidas como *Carrier Threads*).
- Ao contrário das *threads* de plataforma, as virtuais geralmente têm uma pilha de chamadas (*stack*) rasa, sendo ideais para executar tarefas que passam a maior parte do tempo bloqueadas — como aguardar o retorno de uma chamada HTTP ou uma consulta JDBC.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

2.4. Virtual Threads



Adoptar em aplicações com uso intensivo de input/output como clientes HTTP, consultas à bases de dados ou micro-serviços...



Evitar em cálculos de longa duração que consomem muitos recursos de CPU, pois eles podem "fixar" a *thread carrier* e anular os benefícios de desempenho.



Não usar em blocos sincronizados, pois eles podem "fixar" uma *thread virtual* à sua *thread carrier*, impedindo sua desmontagem.



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

// Autor: Celso Paim, adaptado de Pablo Ruiz

// Data: Marco/2026

// Objectivo: demonstrar o uso de virtual threads (so funciona com Java 21 ou superior)

```
public class VirtualThreadsDemo {  
  
    public static void main( String[] args ) throws InterruptedException {  
        Counter count = new Counter();  
  
        Runnable tarefa = () -> {  
            for( int i = 0; i < 1000; i++ ) {  
                count.increment();  
            }  
        };  
  
        Thread t1 = Thread.startVirtualThread( tarefa );  
        Thread t2 = Thread.startVirtualThread( tarefa );  
  
        t1.join();  
        t2.join();  
  
        System.out.println( "Valor final do count =" + count.getCount());  
    }  
}
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

```
}
```

```
class Counter {  
    private int count = 0;  
  
    public void increment() {  
        count++;  
    }  
  
    public int getCount() {  
        return count;  
    }  
}
```



Universidade Católica de Angola

Faculdade de Engenharia Informática

Sistemas Distribuídos e Paralelos I

